

Q1: Using the above equations, decode the actual kinematics (X_{test}) from the neural firing observations (R_{test}). First you should use the training variables to determine the system dynamics (i.e., use X_{train} and R_{train}). Then iterate over each time point of the test set, with the final 'decoded' kinematic variables for that time point corresponding to the a posteriori estimate. Make sure to save these values, so that you can assess how your decoded kinematics matches up with reality (i.e., X_{test}). Use R^2 to assess the goodness of fit of your decoder. What was your R^2 for each kinematic variable? Plot the decoding results for 100 time-steps in the format of slide 12 (lecture pdf).

x position: $R^2 = 0.8915$

y position: $R^2 = 0.9097$

x velocity: $R^2 = 0.6818$

y velocity: $R^2 = 0.6308$

Position variables achieve higher R^2 than velocity variables because they are more directly encoded in the neural population, while velocity is noisier and harder to reconstruct.

Q2: The values of the Kalman gain evolve over time but eventually stabilizes when the uncertainty of the estimates becomes low. Approximately how many time steps does it take for each to stabilize? Plot the Kalman gain for the first 100 or so time steps (as in slide 20 of the lecture pdf). You'll notice that for some kinematic variables the Kalman gain is positive while others it is negative. Please provide a functional explanation of this fact (i.e., Why would this be the case?). Note: There is one Kalman gain for every neuron-kinematics pair (i.e., there are 5×113 of them!). To get a better grasp on its evolution, take the mean Kalman gain across neurons. This will give you one overall Kalman gain per four kinematic variable (ignore the baseline), which you can plot.

The Kalman gain stabilizes after approximately 15–20 time steps. This occurs because the uncertainty in the state estimate, represented by the covariance matrices, decreases over time as observations are incorporated recursively. Eventually, the system reaches a steady-state where both the state covariance and Kalman gain converge and no longer change significantly.

The sign of the Kalman gain varies across neurons because neurons exhibit directional tuning. Some neurons increase their firing rate for increases in a given kinematic variable, resulting in positive contributions, while others show the opposite relationship, leading to negative contributions.

Q3: Let's explore how the initialization choices affect decoding performance. Initialize your decoder using four combinations of x_0 and P_0 . Set the same value for each kinematic variable. (1) $x_0 = 0$, $P_0 = 1000$; note that these are the initial values I provided you in the mat

file. (2) $x_0 = -100$, $P_0 = 1000$; (3) $x_0 = 0$, $P_0 = 0.01$; and finally, (4) $x_0 = -100$, $P_0 = 0.01$. What affect does each initialization pair have on immediate performance? Are some combinations better/worse than others? For each of the four combinations, plot the decoding results in the same format as in Question 1.

Initial covariance P_0 has a strong effect on early decoding performance. High values of p_0 lead the filter to rely more on observations, allowing it to quickly correct incorrect initial states. In contrast, low p_0 causes the filter to trust its initial estimate too strongly, resulting in slower adaptation.

The worst performance occurs when x_0 is incorrect and p_0 is low, since the filter is both confidently wrong and slow to update.

Overall, high uncertainty improves adaptability, while low uncertainty increases stability but reduces responsiveness.